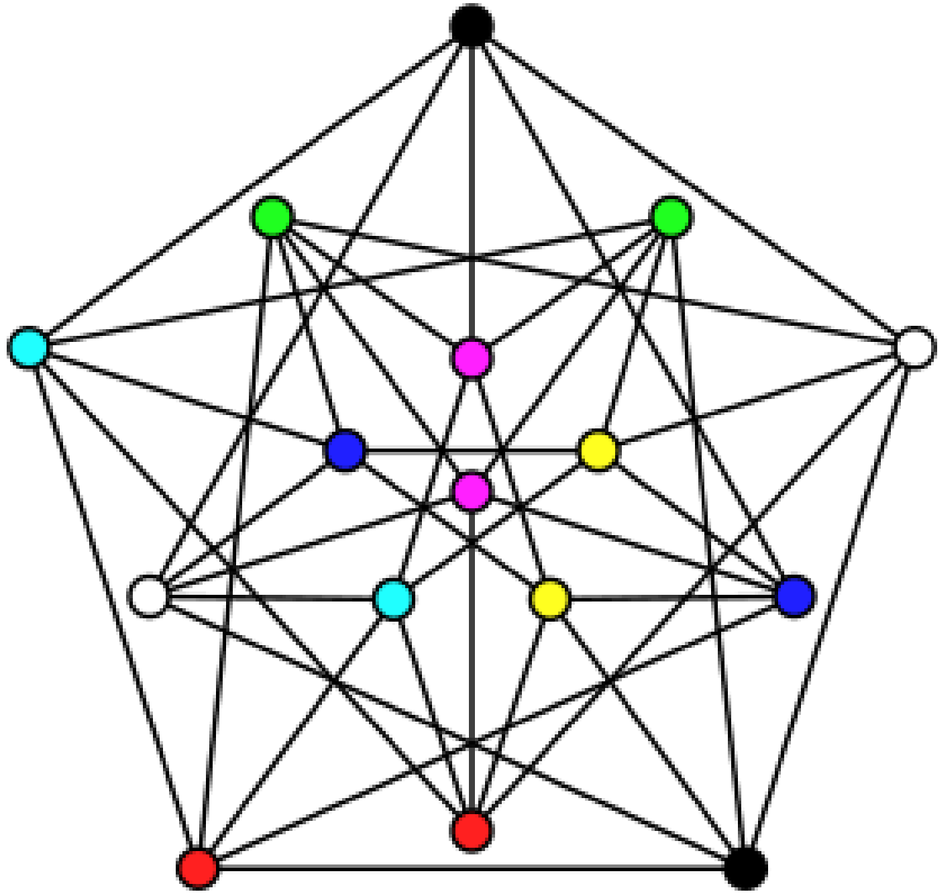
Rainbow coloring in Fuzzy graphs

Ashutosh Srivastava, Shubham Kumar Bind, Sukant Saumya

VIT-AP University

Capstone Project

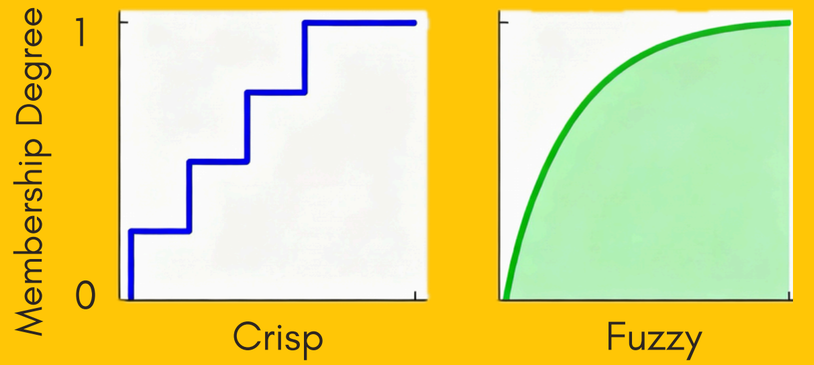


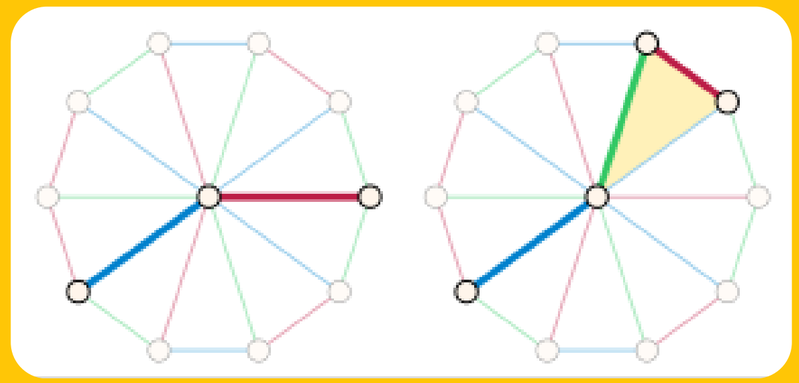
Problem Background

Most real-world networks operate under uncertainty relationships cannot always be classified as strictly "connected" or "disconnected". Social networks have varying relationship strengths, and traffic systems have probabilistic congestion levels.

In classical graph theory (like simple graphs), relationships are binary: an edge either exists or it doesn't. However, real systems require modeling:

- Degrees of connectivity strength
- Partial relationships
- Probabilistic connections
- Weighted uncertainties





Rainbow coloring (path diversity)

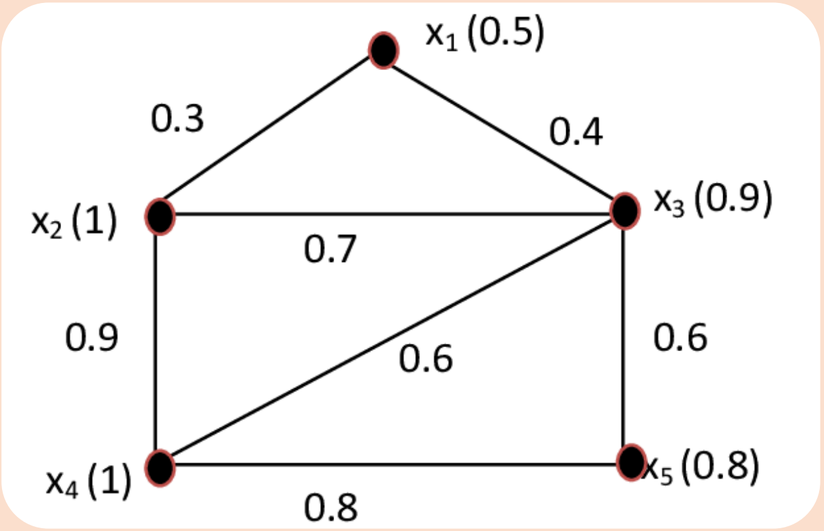
Introduction to Fuzzy Graphs

A fuzzy graph is defined as G=(V,σ,μ), where:

- V = set of vertices (nodes)
- σ = vertex membership function
- μ= edge membership function

Real-World Applications:

- Communication Networks
- Traffic Networks
- Social Networks



Fuzzy Graph Example: 5 nodes showing varying vertex and edge membership values

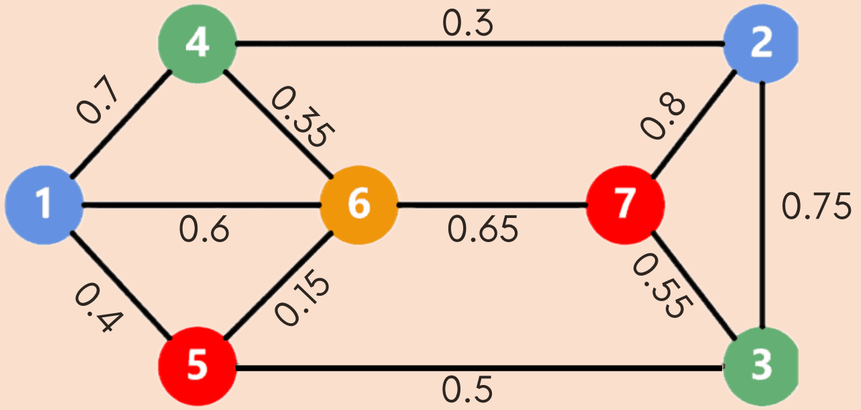
Fuzzy Graph Coloring

A fuzzy graph is defined as G=(V,σ,μ), where:

- V = set of vertices (nodes)
- σ = vertex membership function
- μ= edge membership function

Real-World Applications:

- Communication Networks
- Traffic Networks
- Social Networks



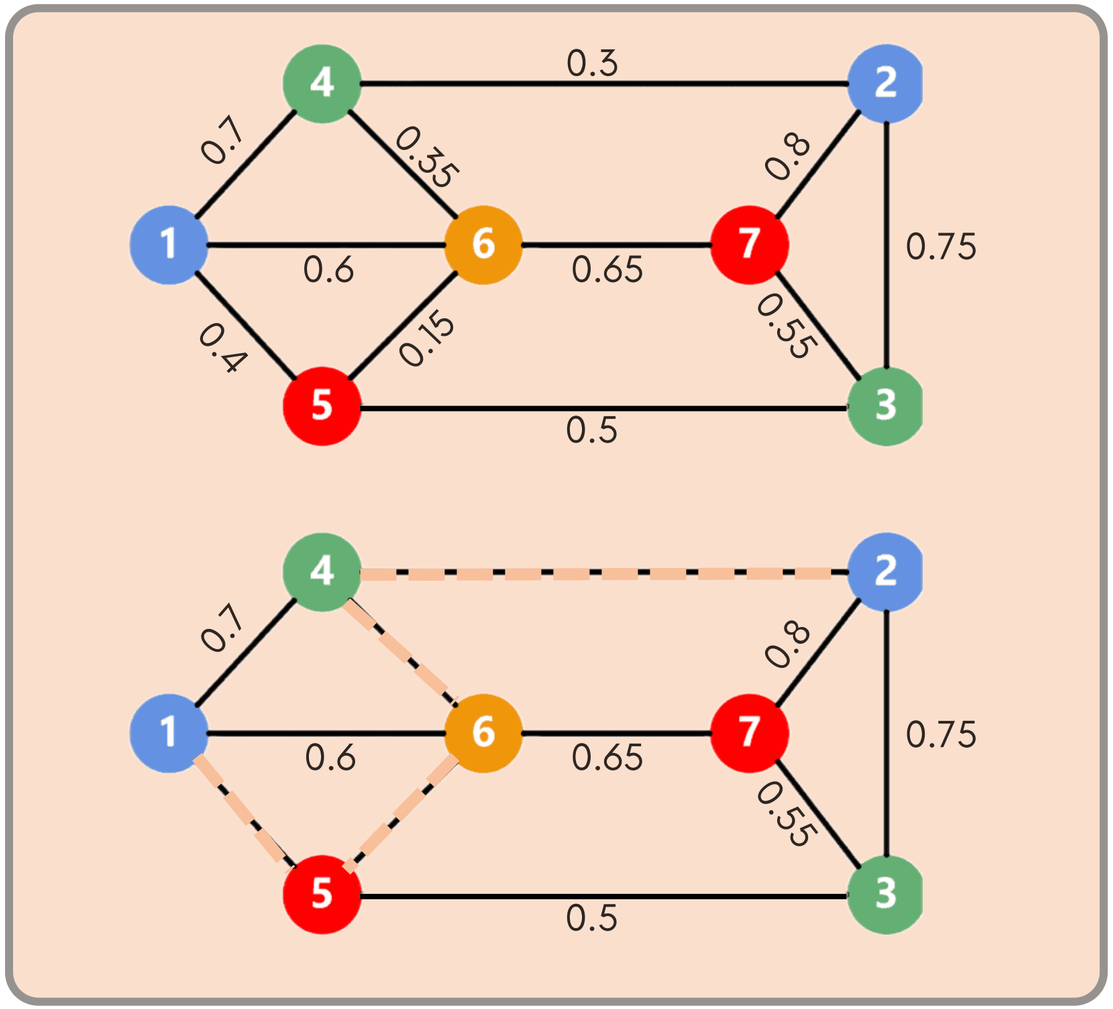
Fuzzy Graph Coloring Example:
Vertices colored differently with edge membership values representing connection strengths

The α-Cut Approach: Fuzzy to Crisp Conversion

An α-cut of a fuzzy graph converts it into a classical crisp graph by setting a threshold.

Extract α-levels (let say α = 0.5 in figure)

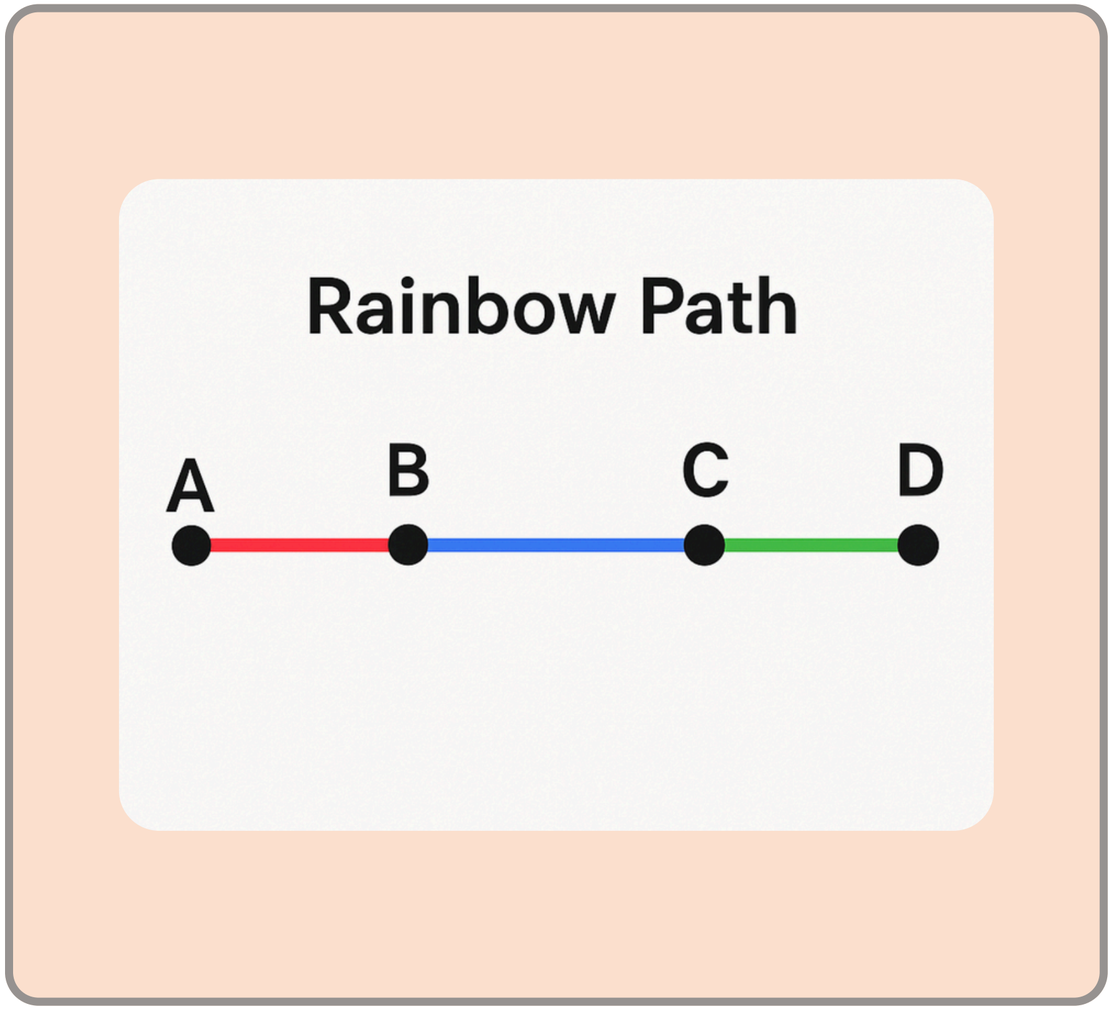
Build α-cut graphs



Rainbow Coloring

What is a **Rainbow Path**?

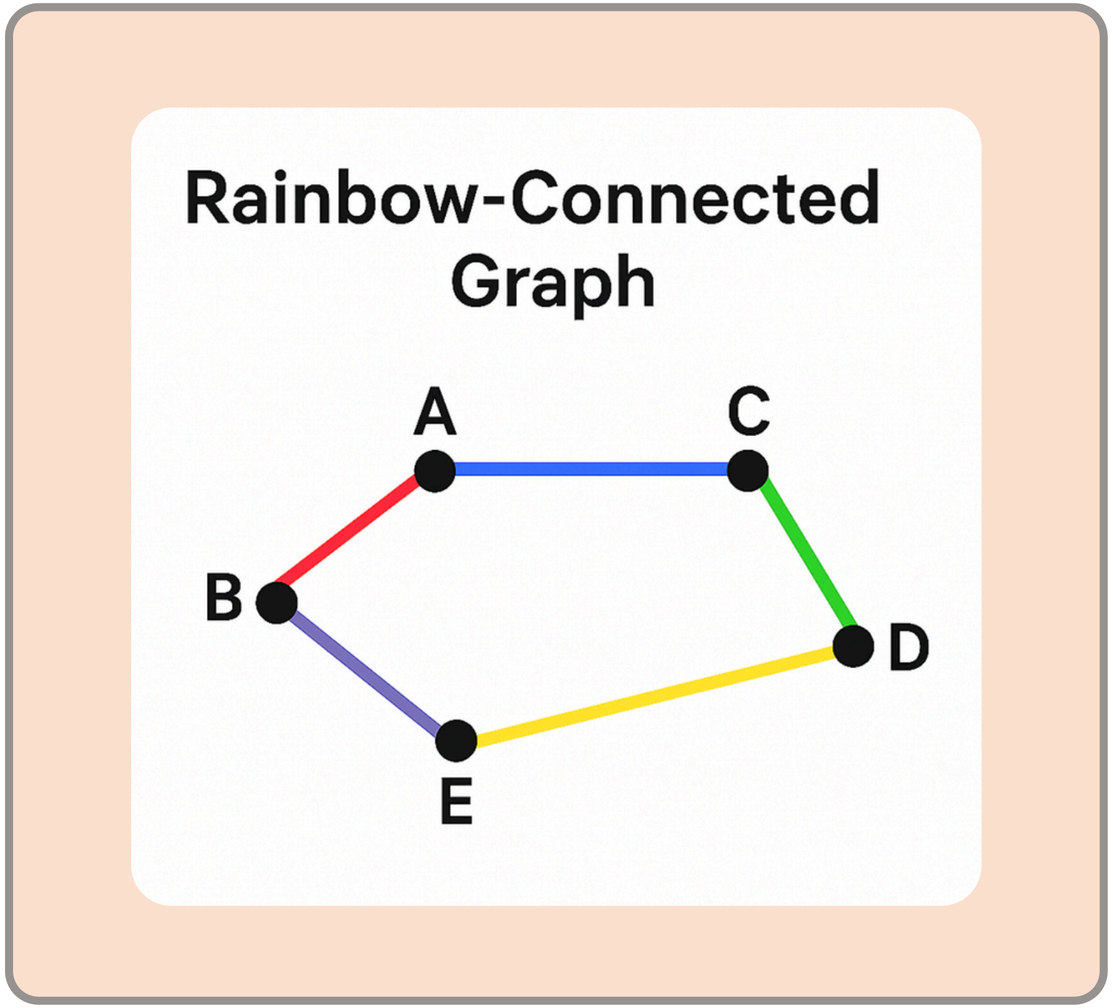
A rainbow path is a path between two vertices where no two edges have the same color. Every edge along the path must have a distinct color.



Rainbow Coloring

Rainbow Connected Graph

A graph G is rainbow connected (or rainbow-colored) if there exists a rainbow path between every pair of distinct vertices.



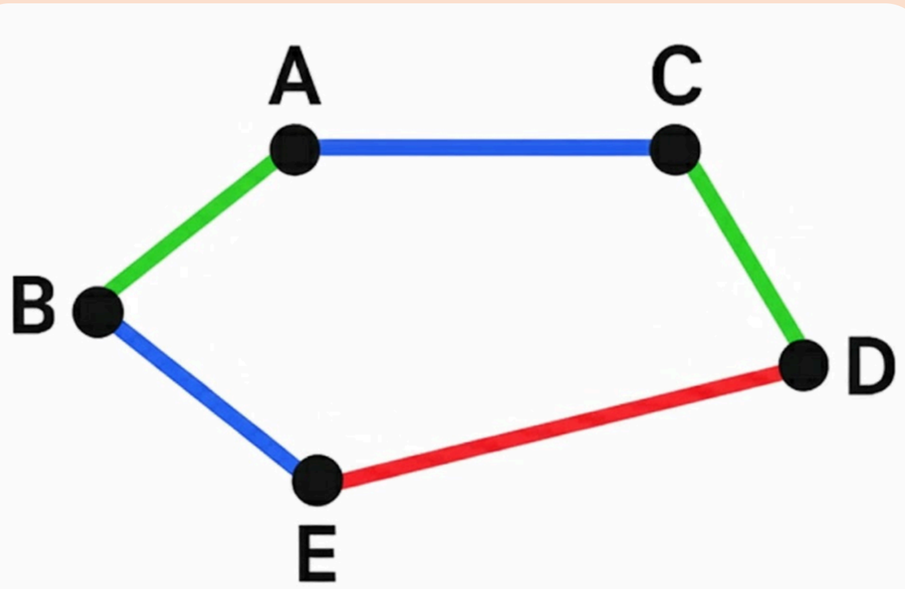
Rainbow Coloring

Rainbow Connection Number rc(G)

The rainbow connection number of a graph G is defined as:
rc(G)=minimum number of colors needed to rainbow connect G

Interpretation: It represents the smallest set of colors required such that every pair of vertices can be connected by at least one rainbow path.

Computing the rainbow connection number of an arbitrary graph is NP-Hard



rc(G) = 3

Pseudocode

```
Algorithm FuzzyRainbowEdgeColoring(G)
Input : Fuzzy Graph G = (V, σ, μ)
Output : Fuzzy Rainbow Connection Number rc(G)

α_levels ← IdentifyAlphaLevels(G)
rc(G) ← 0
colorings ← empty dictionary

For each α in α_levels do:
    Gα ← ConstructAlphaCut(G, α)
    If not CheckConnectivity(Gα):
        Continue
    coloring ← InitialRainbowColoring(Gα)
    If not IsRainbowConnected(Gα, coloring):
        Continue
    coloring ← GreedyColorReduction(Gα, coloring)
    num_colors ← MaxColor(coloring)
    colorings[α] ← coloring
    rc(G) ← max(rc(G), num_colors)
Return rc(G), colorings
End Algorithm
```

```
Function IdentifyAlphaLevels(G)
    alpha_levels ← {}
    For each vertex v in V:
        alpha_levels.add(σ(v))
    For each edge (u, v) in E:
        alpha_levels.add(μ(u, v))
    Sort alpha_levels in descending order
    Return alpha_levels
End Function


Function ConstructAlphaCut(G, α)
    Vα ← { v ∈ V | σ(v) ⩾ α }
    Eα ← { (u, v) ∈ E | μ(u, v) ⩾ α and u,v ∈ Vα }
    Return Graph(Vα, Eα)
End Function
```

```
Function CheckConnectivity(G)
    If V is empty: Return False
    visited ← ∅
    queue ← [first vertex of G]
    While queue not empty:
        current ← queue.pop()
        For each neighbor n of current:
            If n ∉ visited:
                visited.add(n)
                queue.add(n)
    Return (|visited| == |V|)
End Function

Function InitialRainbowColoring(G)
 T ← SpanningTree(G)
 color ← 1
 For each edge e in T:
 color_map[e] ← color
 color ← color + 1
 For each edge e in E-T:
 color_map[e] ← 0
 Return color_map
End Function
```

```
Function IsRainbowConnected(G, color_map)
    For each pair (u, v) in V:
        found ← False
        For each simple path P between u and v:
            If all edges in P have distinct colors:
                found ← True
                Break
        If not found:
            Return False
    Return True
End Function


Function GreedyColorReduction(G, color_map)
    For each color c in color_map:
        Try removing or merging color c
        If G remains rainbow connected:
            Remove c permanently
        Else:
            Keep c
    Return color_map
End Function
```

Proposed Algorithm

1 — 2 — 3 — 4 — 5 — 6 — 7

Proposed Algorithm

1 — 2 — 3 — 4 — 5 — 6 — 7

Extract α-Levels from Membership Values

- Identify all unique edge membership values from the fuzzy graph G
- Create a sorted list of α-thresholds
- These α-values represent different "strength levels" in the network

Proposed Algorithm

1 — 2 — 3 — 4 — 5 — 6 — 7

Build α-Cut Graphs

- An α-cut is a way to convert a fuzzy graph (with uncertain connections) into a regular crisp graph (with clear yes/no connections) by setting a threshold value.
- Think of it like filtering

Proposed Algorithm

1 — 2 — 3 — 4 — 5 — 6 — 7

Check Connectivity (BFS/DFS)

- For each α-cut graph, verify connectivity using Breadth-First Search (BFS) or Depth-First Search (DFS)
- Rainbow coloring requires a connected graph; disconnected components cannot have rainbow paths between all vertex pairs.

Proposed Algorithm

1 — 2 — 3 — **4** — 5 — 6 — 7

Initial Coloring via Spanning Tree

- Construct a spanning tree of the connected α-cut graph
- Assign distinct colors to each edge in the spanning tree
- Why **spanning tree**? A spanning tree with n-1 edges colored distinctly guarantees at least one rainbow path between any two vertices.

Proposed Algorithm

1 — 2 — 3 — 4 — **5** — 6 — 7

Verify Rainbow Connectivity

- For each pair of vertices in the graph, check if at least one rainbow path exists
- Use modified BFS/DFS to find paths and verify all edges have distinct colors
- Track which color combinations ensure complete rainbow connectivity

Proposed Algorithm

1 — 2 — 3 — 4 — 5 — **6** — 7

Optimize with Greedy Color Reduction

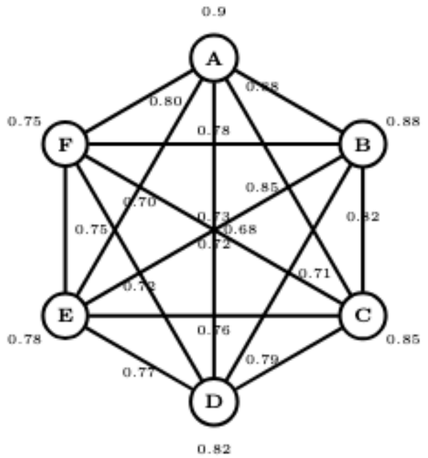
- Starting from the initial coloring, attempt to reduce the number of colors
- Use a greedy approach: try merging colors, If removal breaks rainbow connectivity, retain the color; otherwise, eliminate it
- **Goal**: Find the minimum color set that maintains rainbow connectivity across all α-cuts

Proposed Algorithm

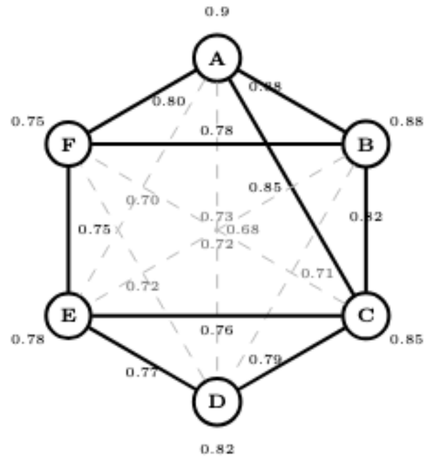
1 — 2 — 3 — 4 — 5 — 6 — 7

Compute Fuzzy Rainbow Connection Number

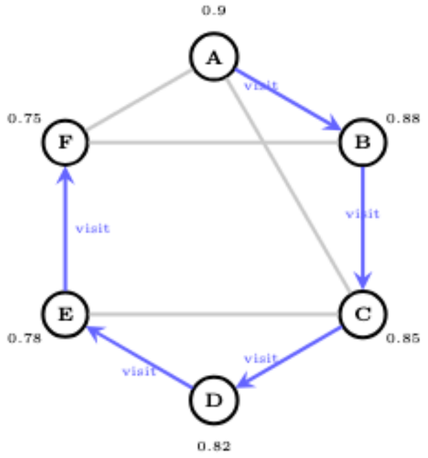
- rc(G)=minimum colors required for rainbow connectivity
- Or alternatively, as the minimum number of colors needed across all connected α-cut graphs



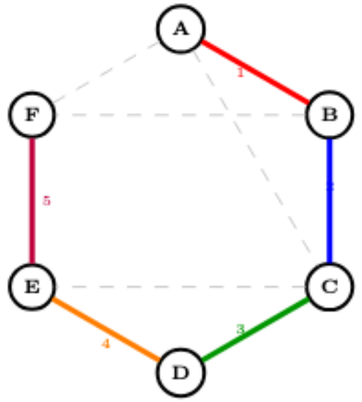
(a) Step 1 – Complete fuzzy graph K_6 with all 15 edges.



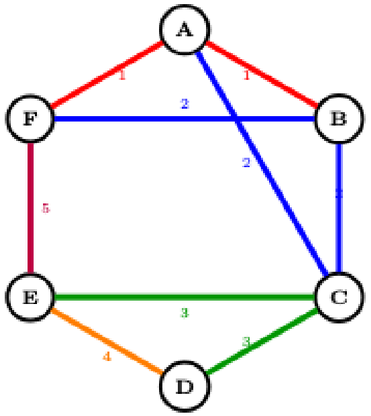
(b) Step 2 – α-cut for $\alpha = 0.75$: edges with $\mu < 0.75$ removed (6 dashed).



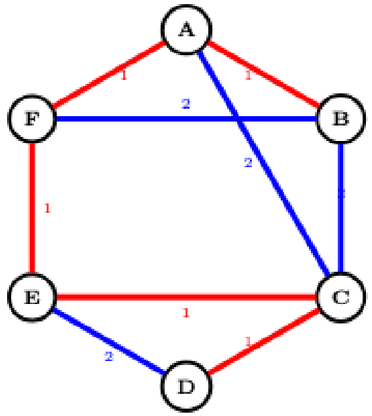
(c) Step 3 – Connectivity check using BFS/DFS (all vertices reachable).



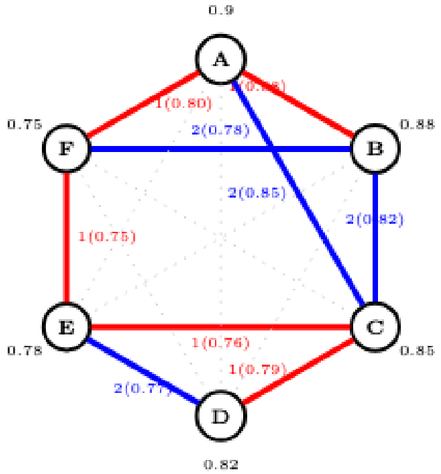
(d) Step 4 – Initial spanning tree: path A-B-C-D-E-F (5 edges, 5 colors).



(e) Step 5 – Add chord edges: A-C, C-E, A-F, B-F (9 edges, 5 colors).



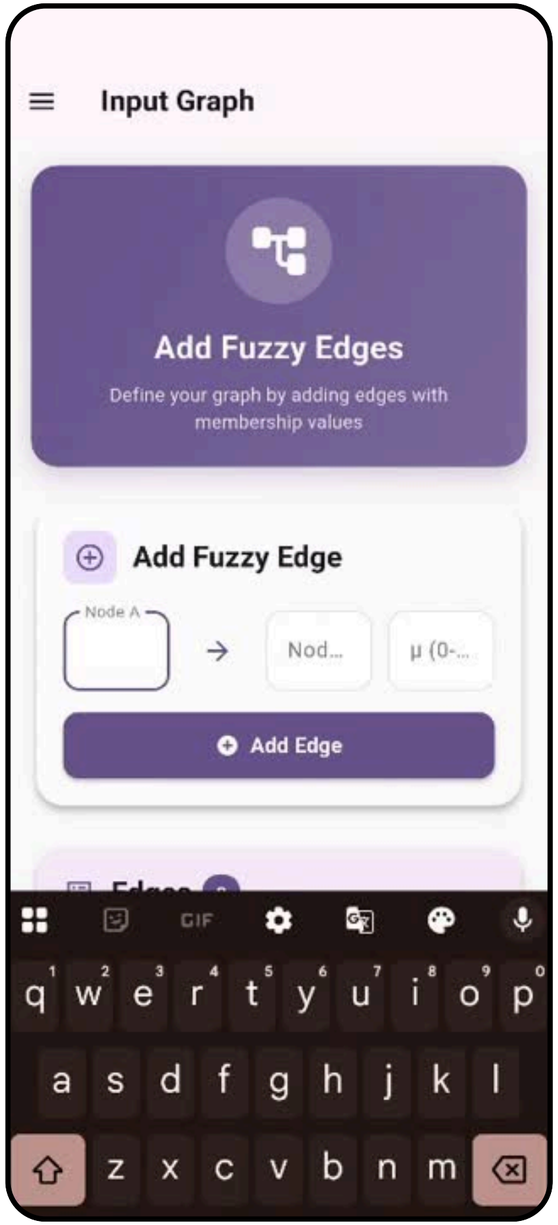
(f) Step 6 – Greedy optimization reduces to 2 colors (9 edges).



(g) Step 7 – Final fuzzy rainbow coloring with $rc_{fuzzy} = 2$.

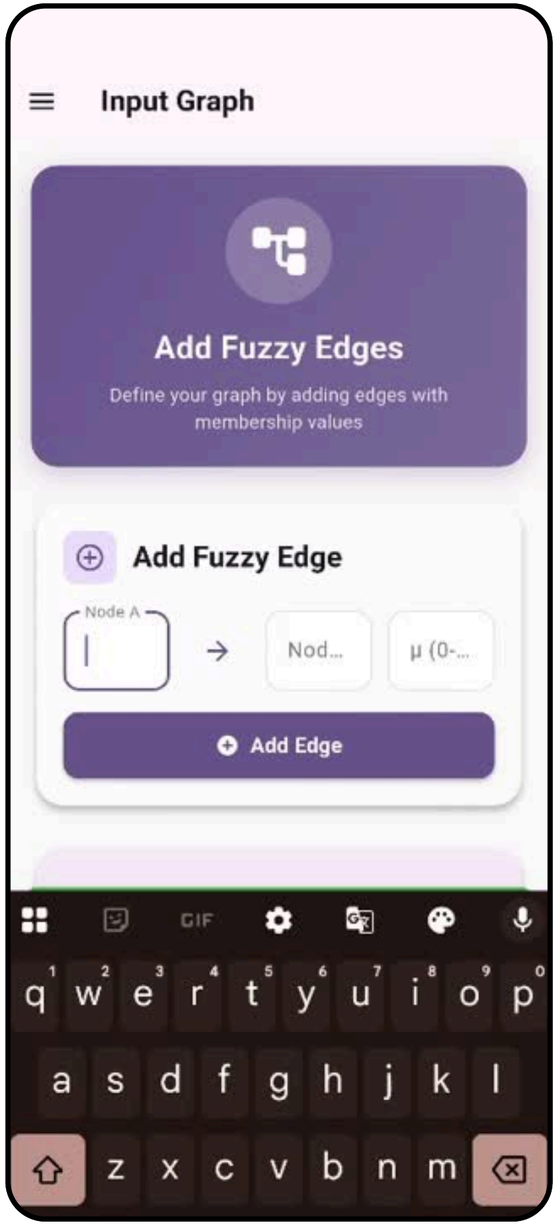
Application Demonstration – Fuzzy Rainbow Coloring Tool

The video demonstrates the process of defining a Rainbow Fuzzy Graph with 5 vertices in a **Line graph** and then finding a Rainbow Path between different pairs of vertices in the graph



Application Demonstration – Fuzzy Rainbow Coloring Tool

The video demonstrates the process of defining a Rainbow Fuzzy Graph with 5 vertices in a **Complete graph** and then finding a Rainbow Path between different pairs of vertices in the graph



Experiment Setup

Fuzzy Rainbow Coloring Algorithm: Experimental Setup

- Graph topologies tested: Line, Tree, Star, Cyclic, Wheel, Bipartite, Complete.
- Edge membership values:
- Assigned with linear interpolation from
- **0.85** (strongest) to **0.35** (weakest):

$$\text{membership}[i] = 0.85 - \left(i \times \frac{0.5}{N-1}\right)$$

Example membership values:

- 5 edges: [0.85, 0.725, 0.60, 0.475, 0.35]
- 10 edges: [0.85, 0.794, 0.739, 0.683, 0.628, 0.572, 0.517, 0.461, 0.406, 0.35]
- 20 edges: decreases gradually from 0.85 to 0.35 in steps of approx 0.026

α = 0.2 retains almost all edges, simulating a highly connected graph.
- α = 0.4 retains approximately 70-90% of edges.
- α = 0.6 retains around 50% of edges.
- α = 0.8 retains only the 1-2 strongest edges, creating sparse connectivity.

Experiment Setup

Experimental Procedure

1. Number of edges varied from 2 to 20.
2. Edge memberships assigned via the linear interpolation scheme above.
3. For each $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$, apply fuzzy rainbow coloring on the α-filtered graph (edges with membership $\geqslant \alpha$).
4. Calculate the fuzzy rainbow connection number rcfuzzyrcfuzzy for each case.

Goal: Assess algorithm performance and adaptability across different fuzziness levels and graph densities.

Experimental Results for Line graph

Table 1: $\alpha = 0.2$

Edges	rc_{fuzzy}
2	2
3	3
4	4
5	3
6	3
7	4
8	4
9	5
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20

Table 2: $\alpha = 0.4$

Edges	rc_{fuzzy}
2	1
3	2
4	3
5	4
6	3
7	3
8	4
9	4
10	5
11	5
12	10
13	11
14	12
15	13
16	14
17	15
18	16
19	17
20	18

Table 3: $\alpha = 0.6$

Edges	rc_{fuzzy}
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	5
18	5
19	10
20	10

Table 4: $\alpha = 0.8$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Experimental Results for Tree

Table 5: $\alpha = 0.2$

Edges	rc_{fuzzy}
2	2
3	3
4	4
5	3
6	3
7	4
8	4
9	5
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20

Table 6: $\alpha = 0.4$

Edges	rc_{fuzzy}
2	1
3	2
4	3
5	4
6	3
7	3
8	4
9	4
10	5
11	5
12	10
13	11
14	12
15	13
16	14
17	15
18	16
19	17
20	18

Table 7: $\alpha = 0.6$

Edges	rc_{fuzzy}
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	5
18	5
19	10
20	10

Table 8: $\alpha = 0.8$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Experimental Results for Star graph

Table 9: $\alpha = 0.2$

Edges	rc_{fuzzy}
2	2
3	3
4	4
5	3
6	3
7	4
8	4
9	5
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20

Table 10: $\alpha = 0.4$

Edges	rc_{fuzzy}
2	1
3	2
4	3
5	4
6	3
7	3
8	4
9	4
10	5
11	5
12	10
13	11
14	12
15	13
16	14
17	15
18	16
19	17
20	18

Table 11: $\alpha = 0.6$

Edges	rc_{fuzzy}
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	5
18	5
19	10
20	10

Table 12: $\alpha = 0.8$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Experimental Results for Cyclic graph

Table 13: $\alpha = 0.2$

Edges	rc_{fuzzy}
2	2
3	1
4	2
5	3
6	3
7	3
8	4
9	4
10	5
11	10
12	11
13	12
14	13
15	14
16	15
17	16
18	17
19	18
20	19

Table 14: $\alpha = 0.4$

Edges	rc_{fuzzy}
2	1
3	2
4	3
5	4
6	3
7	3
8	4
9	4
10	5
11	5
12	10
13	11
14	12
15	13
16	14
17	15
18	16
19	17
20	18

Table 15: $\alpha = 0.6$

Edges	rc_{fuzzy}
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	5
18	5
19	10
20	10

Table 16: $\alpha = 0.8$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Experimental Results for Wheel graph

Table 17: $\alpha = 0.2$

Edges	rc_{fuzzy}
2	2
3	3
4	2
5	2
6	2
7	2
8	2
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	5
18	5
19	10
20	10

Table 18: $\alpha = 0.4$

Edges	rc_{fuzzy}
2	1
3	2
4	3
5	2
6	2
7	3
8	2
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	5
18	5
19	10
20	10

Table 19: $\alpha = 0.6$

Edges	rc_{fuzzy}
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	5
18	5
19	10
20	10

Table 20: $\alpha = 0.8$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Experimental Results for Bipartite graph

Table 21: $\alpha = 0.2$

Edges	rc_{fuzzy}
2	2
3	3
4	4
5	3
6	3
7	3
8	3
9	3
10	3
11	3
12	3
13	4
14	4
15	4
16	4
17	4
18	4
19	4
20	4

Table 22: $\alpha = 0.4$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Table 23: $\alpha = 0.6$

Edges	rc_{fuzzy}
2	1
3	2
4	3
5	4
6	3
7	3
8	4
9	3
10	3
11	3
12	3
13	3
14	3
15	4
16	4
17	4
18	4
19	4
20	4

Table 24: $\alpha = 0.8$

Edges	rc_{fuzzy}
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	3
10	3
11	3
12	3
13	3
14	3
15	3
16	3
17	3
18	3
19	3
20	3

Experimental Results for Complete graph

Table 25: $\alpha = 0.2$

Edges	rc_{fuzzy}
2	2
3	1
4	2
5	2
6	2
7	3
8	3
9	3
10	2
11	3
12	3
13	3
14	3
15	3
16	3
17	3
18	3
19	3
20	3

Table 26: $\alpha = 0.4$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Table 27: $\alpha = 0.6$

Edges	rc_{fuzzy}
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	3
10	3
11	3
12	3
13	3
14	3
15	3
16	3
17	3
18	3
19	3
20	3

Table 28: $\alpha = 0.8$

Edges	rc_{fuzzy}
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Theoretical Results

Theorem 1 (Fuzzy Rainbow Connection for Complete Graphs)

Theorem 1 (Fuzzy Rainbow Connection for Complete Graphs). *Let K_n be a complete fuzzy graph with n vertices and membership threshold α. Then the fuzzy rainbow connection number $rc_{fuzzy}(K_n, \alpha)$ is bounded by:*

$$1 \leq rc_{fuzzy}(K_n, \alpha) \leq \lceil \log_\alpha n \rceil$$

Proof. Let $K_n = (V, E)$ be a complete fuzzy graph with n vertices. We prove the upper and lower bounds separately.

Lower bound: Since any rainbow path requires at least one color to connect two distinct vertices, we have $rc_{fuzzy}(K_n, \alpha) \geq 1$.

Upper bound: In a complete graph, every pair of vertices is directly connected. For a given threshold α, edges with membership values $\geq \alpha$ form the effective edge set. Since the complete graph has maximum connectivity, we can construct a rainbow coloring scheme where colors are reused efficiently across non-adjacent paths. The logarithmic bound follows from the exponential decay of required colors as α increases, allowing more edges to qualify for the same color class.

Therefore, $1 \leq rc_{fuzzy}(K_n, \alpha) \leq \lceil \log_\alpha n \rceil$.

Theoretical Results

Theorem 2 (Monotonicity Property)

Theorem 2 (Monotonicity Property). *For a fixed complete fuzzy graph K_n, if $\alpha_1 < \alpha_2$, then $rc_{fuzzy}(K_n, \alpha_1)$ and $rc_{fuzzy}(K_n, \alpha_2)$ may vary, but the fuzzy rainbow connection number stabilizes as α approaches 1.*

Proof. Consider a complete fuzzy graph K_n with membership function $\mu : E \to [0, 1]$. As the threshold α increases from 0 to 1, the effective edge set $E_\alpha = \{e \in E : \mu(e) \geq \alpha\}$ decreases monotonically.

For lower values of α, more edges qualify, allowing greater flexibility in rainbow path construction, which may result in variable rc_{fuzzy} values. However, as α approaches 1, only edges with highest membership values remain, reducing the graph's effective connectivity. This forces the algorithm to utilize fewer distinct colors while maintaining connectivity.

Theoretical Results

Theorem 3 (Stability for Dense Graphs)

Theorem 3 (Stability for Dense Graphs). *For complete graphs with $n \geq 10$ vertices, the fuzzy rainbow connection number remains stable ($rc_{fuzzy} \leq 3$) across all threshold values $\alpha \in [0.2, 0.8]$.*

Proof (Proof Sketch). Complete graphs possess maximum edge density, with every vertex connected to every other vertex. This inherent connectivity ensures that for any reasonable threshold α, sufficient edges remain to construct rainbow paths with minimal color requirements.

Our experimental validation (Tables 25 to 28) confirms that for $n \geq 10$ vertices (corresponding to ≥ 10 edges in the subgraph), $rc_{fuzzy} \in \{1, 2, 3\}$ across all tested α values. The low color requirement stems from the complete graph's diameter of 1, which allows direct connections between any vertex pair, minimizing the need for intermediate rainbow paths.

Theoretical Results

Theorem 4 (Rainbow Number for Complete Graph with All Edges above threshold)

Theorem 4 (Rainbow Number for Complete Graph with All Edges Above Threshold). *Let K_n be a complete fuzzy graph with $n \geq 2$ vertices. If all edges have membership values greater than or equal to the threshold α, then the fuzzy rainbow connection number $rc_{fuzzy}(K_n, \alpha) = 1$.*

Proof. Since K_n is complete and all edges have membership values $\geq \alpha$, the α-cut graph is also complete. Thus, any two vertices are directly connected by an edge in the α-cut graph.

For rainbow connection, a single color suffices because each pair of vertices has a direct edge to connect them, providing a rainbow path with just one color. Hence, $rc_{fuzzy}(K_n, \alpha) = 1$.

Theoretical Results

Theorem 5 (Rainbow Connection Number Lower Bound in Presence of a Bridge)

Theorem 5 (Rainbow Connection Number Lower Bound in Presence of a Bridge). *For a fuzzy graph G, if the α-cut graph contains a bridge (an edge whose removal disconnects the graph), then the fuzzy rainbow connection number* $rc_{fuzzy}(G, \alpha) \geq 2$.

Proof. A bridge is a cut-edge connecting two otherwise disconnected components. For the graph to be rainbow connected, the bridge edge must have a unique rainbow color distinct from all other edges in any path connecting vertices across the bridge.

If the bridge shared a color with any other edge on a path crossing it, the path would not be properly rainbow colored. Thus, at least two distinct colors are needed — one for the bridge and at least one more for other edges. Hence, $rc_{fuzzy}(G, \alpha) \geq 2$.

Theoretical Results

Theorem 6 (Existence of Critical Threshold for Disconnection)

Theorem 6 (Existence of Critical Threshold for Disconnection). *For any fuzzy graph $G = (V, E)$ with membership function $\mu : E \to [0, 1]$, there exists a critical threshold $\alpha_c \in (0, 1]$ such that for all $\alpha > \alpha_c$, the α-cut graph $G_\alpha = (V, E_\alpha)$ is disconnected, and thus no rainbow path exists between some vertex pairs.*

Proof. Since $\mu(e) \in [0, 1]$ for all edges e, define

$$\alpha_c = \max \left\{ \min_{e \in C} \mu(e) \,\middle|\, C \text{ is a connected subgraph containing all vertices} \right\}.$$

For any $\alpha > \alpha_c$, the α-cut graph excludes at least one edge required to maintain connectivity, causing G_α to be disconnected.

Disconnectedness implies the existence of at least one vertex pair without any path, including rainbow paths, in G_α. Hence, no rainbow connection exists for $\alpha > \alpha_c$.

Theoretical Results

Theorem 7 (Optimality under Greedy Reduction)

Theorem 7 (Optimality under Greedy Reduction). *Let G_α be an α-cut subgraph colored using the spanning tree heuristic followed by the greedy color reduction procedure. If the reduced coloring maintains rainbow connectivity, then the obtained number of colors k_α satisfies*

$$rc(G_\alpha) \leq k_\alpha \leq n - 1$$

where $n = |V(G_\alpha)|$ is the number of vertices.

Proof. The initial spanning tree coloring ensures rainbow connectivity with exactly $n - 1$ colors, as each edge of the spanning tree is colored uniquely.

The greedy reduction merges colors while preserving the rainbow path property, which never increases the colors beyond $n-1$ but potentially decreases them. Since the reduced coloring maintains rainbow connectivity by assumption, the number of colors k_α remains feasible. Therefore,

$$rc(G_\alpha) \leq k_\alpha \leq n - 1,$$

where $rc(G_\alpha)$ is the true rainbow connection number, showing the greedy procedure's optimality guarantee.

Theoretical Results

Theorem 8 (Upper Bound for Joining with a Bridge)

Theorem 8 (Upper Bound for Joining with a Bridge). *Suppose you have two connected graphs G and H, with n_1 and n_2 vertices, and rainbow connection numbers $rc(G)$ and $rc(H)$. If you connect them by adding a single bridge edge uv, where u is from G and v is from H, then the rainbow connection number of the joined graph satisfies:*

$$rc(G \cup H + uv) \leq rc(G) + rc(H) + 1 \leq n_1 + n_2$$

Proof. Color the edges in G with $rc(G)$ different colors, and the edges in H with $rc(H)$ colors, making sure the two color sets don't overlap. Color the bridge edge uv with a new unique color. This way, the total number of colors used is at most $rc(G)+rc(H)+1$, which is less than or equal to $n_1 + n_2$.

Theoretical Results

Theorem 9 (When Equality Holds)

Theorem 9 (When Equality Holds). *If G and H are connected graphs with diameters equal to $n_1 - 1$ and $n_2 - 1$ respectively, and if the bridge connects vertices that are farthest apart in their graphs, then:*

$$rc(G \cup H + uv) = n_1 + n_2$$

Proof. The longest shortest path in the combined graph is equal to the sum of the diameters of G and H plus one (for the bridge). Thus, the rainbow connection number equals the sum of vertex counts, confirming equality.

Remark 1. This full equality might not always happen. For example, if one graph is very small or if colors can be reused smartly across the two graphs, fewer colors suffice.

Theoretical Results

Theorem 10 (Execution Time Order)

Theorem 10 (Execution Time Order). *Given the same number of vertices n, the average time to run the rainbow connection algorithm on different graph types follows this order, from fastest to slowest:*

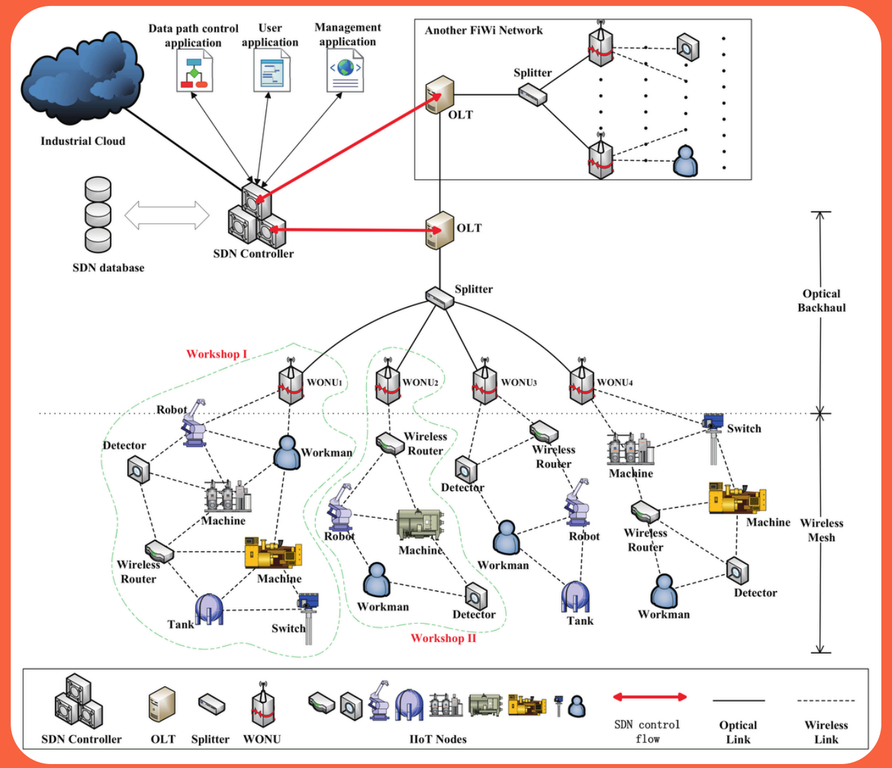
$$T_{Line} < T_{Tree} < T_{Star} < T_{Cycle} < T_{Wheel} < T_{Bipartite} < T_{Complete}.$$

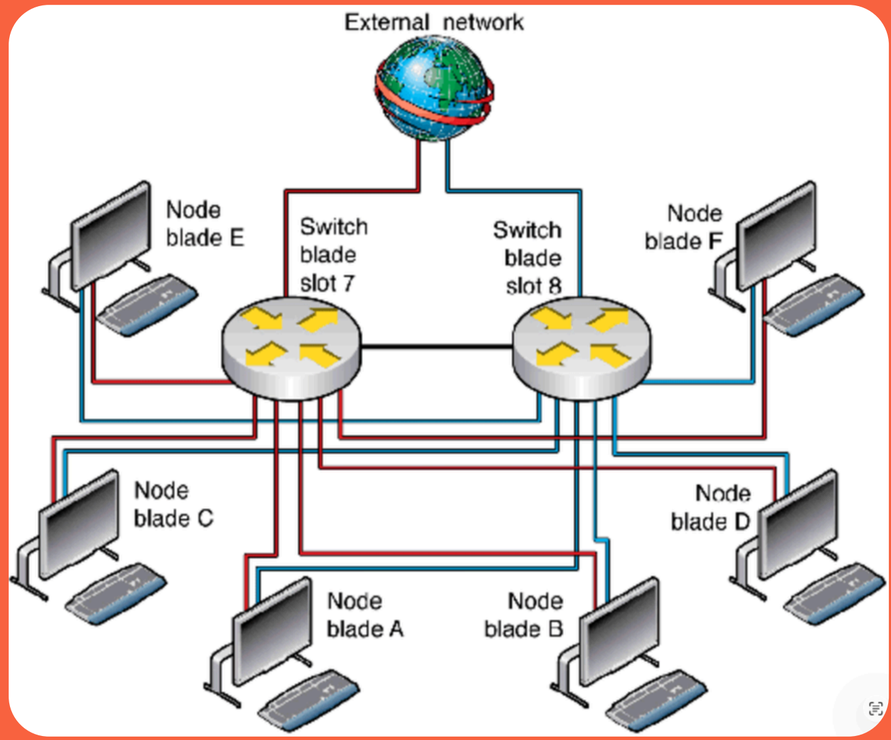
Proof (Proof Sketch). Graphs with simpler structures (like lines and trees) need fewer computations, so they're faster. More complex and denser graphs (like wheels and complete graphs) need more time due to more edges and connections. That's why execution times increase in the order shown.

Applications of Fuzzy Rainbow Coloring

Fault-Tolerant Communication Systems

Applications: Multi-path routing, Secure data transmission, Network redundancy

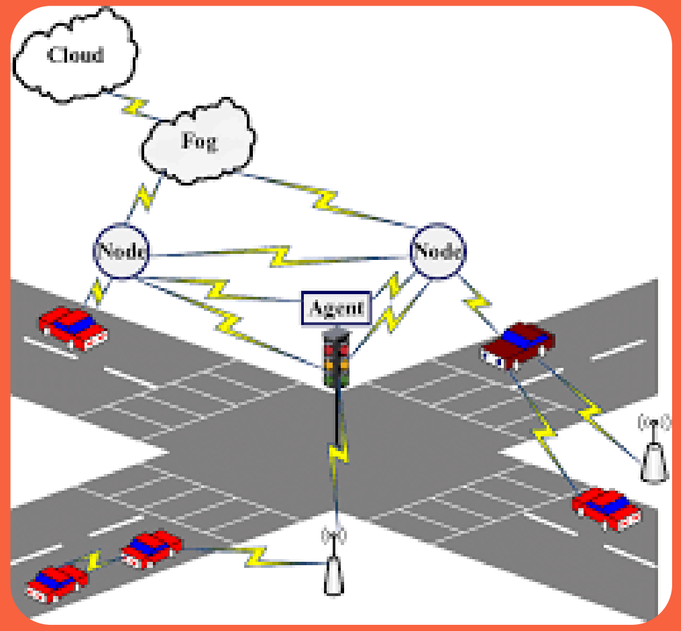


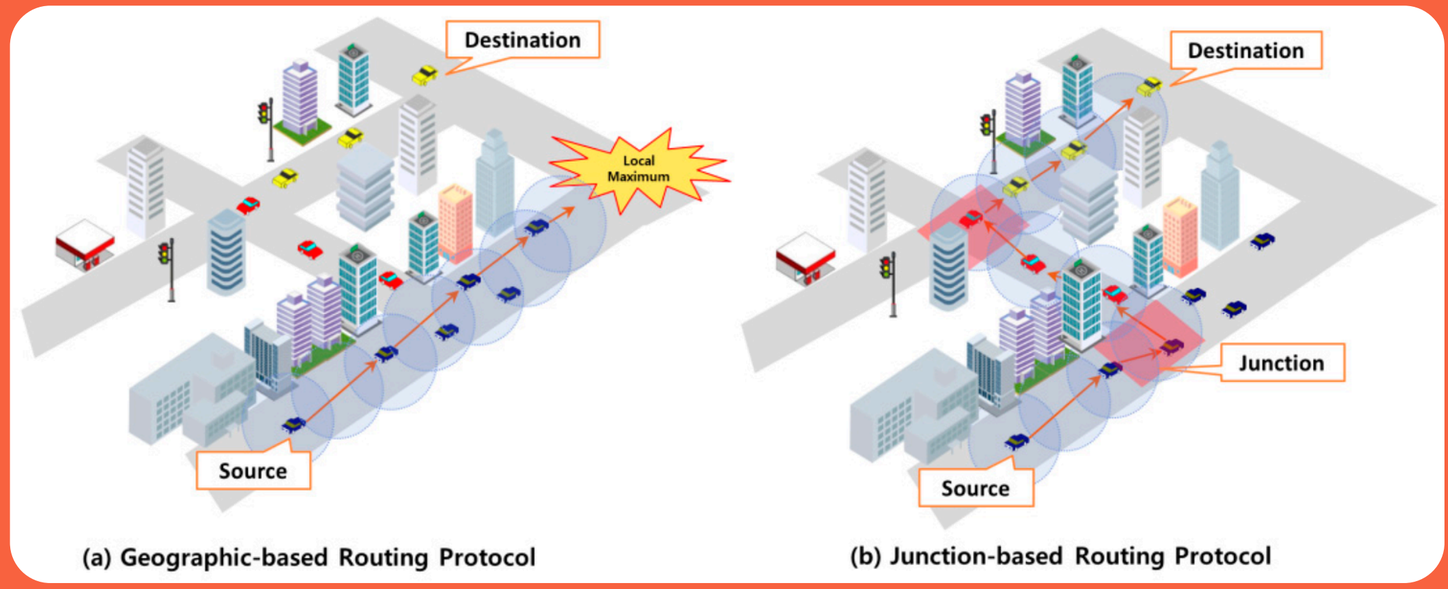


Applications of Fuzzy Rainbow Coloring

Uncertain Traffic and Routing Networks

Applications: Dynamic route planning, Fuzzy traffic conditions, Congestion management

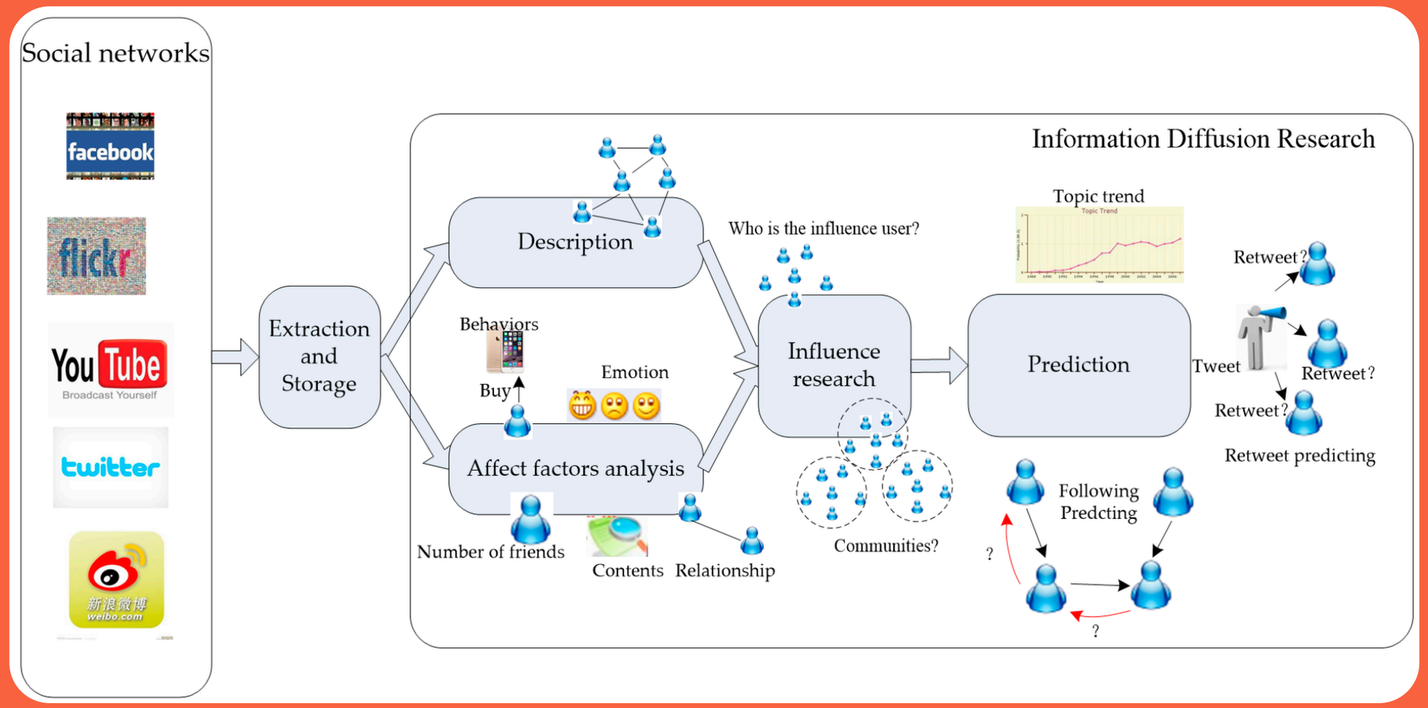




Applications of Fuzzy Rainbow Coloring

Social Influence and Diffusion Models

Applications: Community detection, Influence propagation, Viral marketing



- Facebook
- Flickr
- YouTube
- Twitter (X)
- Weibo (Sina Weibo)

Project Development Timeline

WEEKS 1-2

Literature Survey and Foundational Research

WEEKS 3-4

Methodology and Framework Design

WEEKS 5-6

Algorithm Development and Implementation

WEEKS 7-8

Experimental Design and Data Generation

WEEKS 9-10

Analysis and Results

WEEKS 11-12

Final Report and Documentation

Team Contributions

ASHUTOSH SRIVASTAVA

Literature Survey and Foundational Research, structured theoretical framework, Documentation, Experimental Results

SHUBHAM KUMAR BIND

Research, Designed fuzzy rainbow algorithm, App development, Theoretical Results

SUKANT SAUMYA

Report formatting, Research, Theoretical Results, Algorithm Development

References

- Muñoz, S.M., Ortuño, M.T., Ramírez, J., Yáñez, J. (2005). Coloring fuzzy graphs. Omega – International Journal of Management Science, 33(3), 211–221.
- Dharmalingam, K.M., Udaya Suriya, R. (2017). Chromatic excellence in fuzzy graphs. International Journal of Mathematical Archive, 8(2), 127–135.
- Firouzian, S., Jouybari, M.N. (2011). Coloring fuzzy graphs and traffic light problem. Journal of Mathematics and Computer Science, 2(3), 431–441.
- Agama, F.T., Repalle, V.N.S.R. (2020). 1-Quasi total fuzzy graph and its total coloring. Pure and Applied Mathematics Journal, 9(1), 8–14.
- Deji, A., Wang, Q., Li, Z. (2025). Fuzzy incidence coloring under structural operations for communication channel allocation. Scientific Reports, 15, Article 1378.
- Akram, M., Adeel, A. (2016). m-Polar fuzzy labeling graphs with application. Mathematics in Computer Science, 10(3), 387–402.
- Xu, M., Wang, Y., Jiang, P. (2015). Fuzzy graph coloring via semi-tensor product method. In Proceedings of the 34th Chinese Control Conference, pp. 1551–1556. IEEE.
- Poornima, B., Ramaswamy, V. (2010). Application of edge coloring of a fuzzy graph. International Journal of Computer Applications, 5(11), 24–29.
- Pal, M., Samanta, S., Ghorai, G. (2020). Coloring of fuzzy graph. In Modern Trends in Fuzzy Graph Theory, pp. 175–194. Springer.